



Queues Management Application Using Threads and Synchronization Mechanisms - Assignment 2

Fundamental Programming Techniques, Computer Science Department, TUCN

Spring 2026

1 Requirements

Goal: Design and implement a queues management application which assigns clients to queues such that the waiting time is minimized.

Queues are commonly used to model real world domains. The main objective of a queue is to provide a place for a "*client*" to wait before receiving a "*service*". The management of queue-based systems is interested in minimizing the time amount their "*clients*" are waiting in queues before they are served. One way to minimize the waiting time is to add more servers, i.e., more queues in the system (each queue is considered as having an associated processor) but this approach increases the costs of the service supplier.

The **Queues Management** application should simulate (by defining a simulation time $t_{simulation}$) a series of N clients arriving for service, entering Q queues, waiting, being served and finally leaving the queues. All clients are generated when the simulation is started, and are characterized by 3 parameters: ID - a number between 1 and N , $t_{arrival}$ - simulation time when they are ready to enter the queue and $t_{service}$ - time interval or duration needed to serve the client; i.e. waiting time when the client is in front of the queue. The application tracks the total time spent by every client in the queues and computes the average waiting time. Each client is added to the queue with the minimum waiting time when its $t_{arrival}$ time is greater than or equal to the simulation time $t_{arrival} \geq t_{simulation}$.

The following data should be considered as **input data** for the application that should be inserted by the user in the application's user interface:

1. Number of clients N ;
2. Number of queues Q ;
3. Simulation interval $t_{simulation}^{MAX}$;
4. Minimum and maximum arrival time $t_{arrival}^{MIN} \leq t_{arrival} \leq t_{arrival}^{MAX}$;
5. Minimum and maximum service time $t_{service}^{MIN} \leq t_{service} \leq t_{service}^{MAX}$;

1.1 Example

Consider the following input data for the application:

- $N = 4$ clients
- $Q = 2$ queues
- $t_{simulation}^{MAX} = 60$, thus a 60 seconds simulation
- $[2, 30]$ - the bounds for the client parameters, respectively a minimum and a maximum arrival time, meaning that clients will go to the queues from second 2 up to second 30
- $[2, 4]$ - the bounds for the service time, meaning that a client has a minimum time to wait in front of the queue of 2 seconds and a maximum time of 4 seconds

Using this input data, a set of 4 clients are generated random, each client i being defined by the following tuple: $(ID^i, t_{arrival}^i, t_{service}^i)$ with the following constraints:

1. $1 \leq ID^i \leq N$
2. $t_{arrival}^{MIN} \leq t_{arrival}^i \leq t_{arrival}^{MAX}$



$$3. t_{service}^{MIN} \leq t_{service}^i \leq t_{service}^{MAX}$$

A number of Q threads will be launched to process in parallel the clients. Another thread launched to hold the simulation time $t_{simulation}$ and distribute each client i to the queue with the smallest time when $t_{arrival}^i \geq t_{simulation}$.

The log of events contains the status of the pool of waiting clients and the queues as the simulation time $t_{simulation}$ goes from 0 to $t_{simulation}^{MAX}$. An example of data displayed in the log of events is given in the table below:

Log of events	Explanation
Time 0 Waiting clients: (1, 2, 2), (2, 3, 3), (3, 4, 3), (4, 10, 2) Queue 1: closed Queue 2: closed	At time $t_{simulation} = 0$, a number of 4 clients are generated. Client with $ID = 1$ has arrival time equal to 2, meaning that it will be ready to go to a queue when $t_{simulation} \geq 2$. Furthermore, it has a service time equal to 2, meaning that it needs to stay 2 timestamps in the front of the queue. The same rules apply for the next 3 clients. The two queues are closed since there are no clients available for service yet.
Time 1 Waiting clients: (1, 2, 2), (2, 3, 3), (3, 4, 3), (4, 10, 2) Queue 1: closed Queue 2: closed	At time $t_{simulation} = 1$, none of the clients can be sent to the queues because none has $t_{arrival} \leq t_{simulation}$. The two queues remain closed since there are no eligible clients.
Time 2 Waiting clients: (2, 3, 3), (3, 4, 3), (4, 10, 2) Queue 1: (1, 2, 2) Queue 2: closed	Queue 1 opens and client $ID = 1$ is sent to the first queue since $t_{arrival}^1 \leq t_{simulation} = 2$. Other clients are still waiting. Queue 2 remains closed.
Time 3 Waiting clients: (3, 4, 3), (4, 10, 2) Queue 1: (1, 2, 1) Queue 2: (2, 3, 3)	Queue 2 opens at time $t_{simulation} = 3$. Client $ID = 2$ is sent to Queue 2 since $t_{arrival}^2 \leq t_{simulation}$, and the waiting time at Queue 2 (0) is smaller than the waiting time at Queue 1 (1), where a client is still processed. The client in Queue 1 has its service time decreased to 1 (highlighted) because it is being processed. Other clients are still waiting.
Time 4 Waiting clients: (4, 10, 2) Queue 1: (3, 4, 3) Queue 2: (2, 3, 2)	At time $t_{simulation} = 4$, client $ID = 3$ is sent to Queue 1 since $t_{arrival}^3 \leq t_{simulation}$. Client $ID = 1$ is eliminated from Queue 1 because its service time dropped to 0 (it was 1 at the previous iteration and was decreased by one at this simulation step). The client from Queue 2 has its service time decreased to 2 (highlighted) because it is being processed. The final client is still waiting.
Average waiting time: 2.5	The simulation is finished when there are no more clients in the waiting pool or in the service queues, or when $t_{simulation} > t_{simulation}^{MAX}$. The average waiting time is computed and appended to the log of events.



2 Deliverables

1. **Source code files:** will be uploaded on the personal **Gitlab** account created according to the instructions in the **Laboratory Resources** document, and following steps:
 - (a) Create a private repository on **Gitlab** named according to the following template
PT2026_Group_LastName_FirstName_Assignment_2
 - (b) Push the source code (**not as archive**).
 - (c) Share the repository with the user **utcn_dsrl** with Maintainer as role.
2. The **draw.io** file with the UML use case diagram, package diagram and the class diagram will be uploaded in the repository with the source code files.

3 Evaluation

The assignment will be graded as follows:

Requirements	Grading
Minimum to pass: 1. Object-oriented programming desing 2. Random client generator 3. Multithreading: one thread per queue 4. Appropriate synchronized data structures to assure thread safety 5. Log of events displayed in a .txt file like in the above example 1.1 6. Implement classes with maximum 300 lines (except UI classes) and methods with maximum 30 lines 7. Use the Java naming conventions 8. Correct UML diagrams	5 points
Graphical user interface for: (1) simulation setup, and (2) displaying the real-time queue evolution	+3 point
Display of simulation results (average waiting time, average service time, peak hour for the simulation interval) in the graphical user interface/.txt file corresponding to the log events.	+1 point
Run the application on the input data sets listed in the table below and upload the generated logs of events in your repository.	+1 point

Test 1	Test 2	Test 3
$N = 4$ $Q = 2$ $t_{simulation}^{MAX} = 60$ seconds $[t_{arrival}^{MIN}, t_{arrival}^{MAX}] = [2, 30]$ $[t_{service}^{MIN}, t_{service}^{MAX}] = [2, 4]$	$N = 50$ $Q = 5$ $t_{simulation}^{MAX} = 60$ seconds $[t_{arrival}^{MIN}, t_{arrival}^{MAX}] = [2, 40]$ $[t_{service}^{MIN}, t_{service}^{MAX}] = [1, 7]$	$N = 1000$ $Q = 20$ $t_{simulation}^{MAX} = 200$ seconds $[t_{arrival}^{MIN}, t_{arrival}^{MAX}] = [10, 100]$ $[t_{service}^{MIN}, t_{service}^{MAX}] = [3, 9]$

4 Bibliography

1. **Concurrency:** <https://docs.oracle.com/javase/tutorial/essential/concurrency/index.html>
2. **Java concurrency:** <https://www.baeldung.com/java-concurrency>